#5

Top K Frequent Elements

Arrays & Hashing | LeetCode 347 | NeetCode "Top K Elements in List"

PROBLEM

Given an integer array `nums` and an integer `k`, return the `k` most frequent elements within the array. The test cases are generated such that the answer is always unique. You may return the output in any order.

`nums = [1,2,2,3,3,3], k = 2 → [2,3]`

`nums = [7,7], k = 1 → [7]`

CORE IDEA

Goal

Return the k values that appear most often in an array, without sorting the whole frequency distribution.

Key insight

A value's frequency can never exceed the length of the array, so instead of sorting all (value, frequency) pairs by frequency — O(n log n) — bucket each value directly into an index equal to its frequency count. Reading the buckets from the highest index down then visits values in descending frequency order for free, in O(n) overall.

> Trigger: “Top / most frequent k” with no need for full sorted order → bucket sort by frequency (Hash Map + array of buckets), not a heap or a full sort.

OPTIMAL SOLUTION

Data structure — Hash Map for counts, plus a bucket array of size `len(nums) + 1` indexed by frequency

1. Count every value's frequency into a hash map `freq` with a single pass over `nums`.
2. Create a bucket array `res` of size `len(nums) + 1` — a value can appear at most `len(nums)` times, so every possible frequency has a slot.
3. For each `(key, value)` pair in `freq` (`value` is the frequency), append `key` into `res[value]` — this groups all values that share the same frequency into the same bucket.
4. Walk `res` in reverse (highest frequency first). Any non-empty bucket holds values of that frequency; append them to the output until it has `k` elements.
5. Return the collected output.

O(n)	O(n)
TIME	SPACE

CODE

```python
class Solution:

    def topKFrequent(self, nums: List[int], k: int) -> List[int]:

        freq = defaultdict(int)

        for num in nums:

            freq[num] += 1


        res = [0] * (len(nums)+1)


        for key,value in freq.items():

            if res[value] == 0:

                res[value] = [key]

            else:

                res[value].append(key)


        out = []

        for item in reversed(res):

            if item != 0 and len(out) < k:

                for i in item:

                    out.append(i)


        return out
```

TIPS TO REMEMBER

Mental shortcut — “Frequency can't exceed array length → bucket by frequency, read buckets top-down.”

Common / related — Group Anagrams, Valid Anagram, Contains Duplicate

> *"Frequencies are bounded by the array length, so I bucket values by frequency into a fixed-size array instead of sorting (value, frequency) pairs — that turns an O(n log n) sort into an O(n) counting pass plus an O(n) bucket scan."*

EDGE CASES

- k equals the number of distinct values → every bucket with any value is collected, output has exactly k elements
- Every element is unique → every value sits in bucket index 1; any k of them is a valid answer since the problem guarantees a unique result set
- Empty nums array → freq and buckets stay empty, output is empty (k would be 0 in that case per the problem's constraints)
- All elements are identical, e.g. [7,7] → one bucket at the max index holding a single key, k = 1 returns that key
- Multiple keys landing in the same bucket when k falls in the middle of that bucket: the reference solution's outer check (`len(out) < k`) is tested once per bucket, so it can append an entire tied bucket even if that slightly overshoots k — worth mentioning as a trade-off versus trimming to exactly k inside the inner loop